Jenkins-Based Smart Irrigation Alert Dashboard

A simulated multi-zone monitoring prototype with low-moisture alerts and an academic DevOps workflow

CHETAN KATKAR | 23102A0024

BE CMPN-A | Vidyalankar Institute of Technology

THE PROBLEM

Manual field checks and timer-based irrigation can delay response to dry soil or apply water when zones are already moist.

- Slow checks People must inspect separated fields.
- Water waste Fixed schedules ignore current moisture.
- Scattered readings Managers lack one shared zone view.
- Late action A low reading can go unnoticed between visits.

PROJECT OBJECTIVE

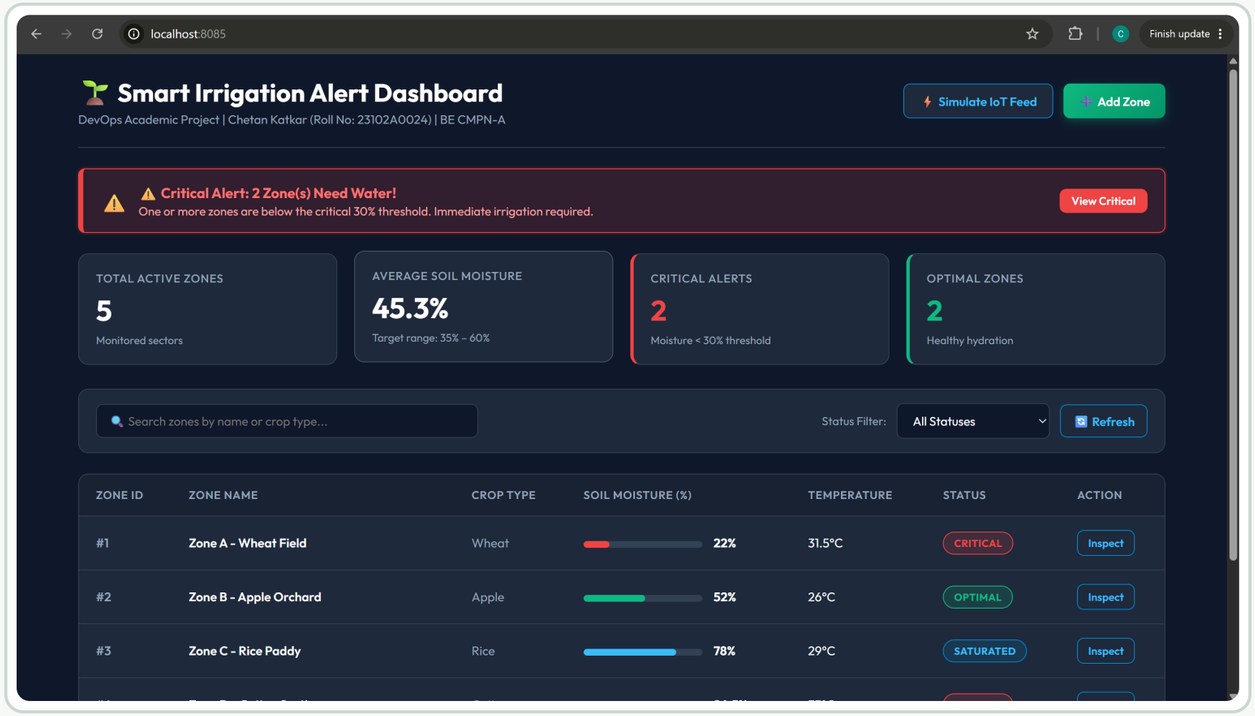
Make simulated moisture and temperature readings visible in one browser dashboard, highlight zones below 30%, and let an operator simulate irrigation.

DEMO DATA

Five seeded zones; readings are simulated, not from physical field sensors.

DASHBOARD PROTOTYPE

Saved local capture from 11 October 2026



Historical screenshot; it documents the interface but does not verify that the Ubuntu application is currently running.

FIVE MVP CAPABILITIES

01 **Zone entry**
Add a named crop zone and initial readings.

02 **Search and filter**
Find by zone/crop and status.

03 **Summary metrics**
Zone count, average moisture, critical and optimal counts.

04 **Zone inspection**
View the current reading and update time.

05 **Critical alerts**
Highlight moisture below 30% and simulate watering.

< 30% CRITICAL MOISTURE
Threshold used by API and dashboard logic

SYSTEM DESIGN

Prototype implementation and documented target workflow

Browser UI → Spring Boot REST → Sensor repository → H2 in-memory DB

DEVOPS DELIVERY WORKFLOW

Target sequence from assignment; live Docker deployment and Ansible idempotency still need current execution evidence.

GitHub → Jenkins → Maven + tests → Docker → Ansible

Source: Task-01 scope and problem statement; current repository source and configuration; saved dashboard screenshot.